Design and Implementation of a Secure Payment Gateway System in C

Sai Teja
Department of Computer Science and Engineering
Email: sau.teja@university.edu

Chandu
Department of Computer Science and Engineering
Email: chandu@university.edu

Abstract

***Abstract*—**This paper presents the design and implementation of a console-based secure payment gateway system developed in the C programming language. The system supports two widely used payment modes: Unified Payments Interface (UPI) and card-based payment. Input validation mechanisms are incorporated to verify UPI ID format and card credential lengths. A simulated transaction processing module is implemented using pseudo-random number generation to model real-world payment success and failure scenarios. The paper discusses the system architecture, control flow, and potential security vulnerabilities along with recommended improvements. This work serves as a foundational prototype for understanding transaction processing pipelines in embedded and low-level software environments.

***Keywords*—**Payment Gateway, UPI, Card Payment, C Programming, Input Validation, Transaction Processing, Security.

I. INTRODUCTION

Digital payment systems have become an integral part of modern commerce, enabling fast and secure transfer of funds across various platforms. Payment gateways serve as the intermediary layer between a customer and the financial institution, validating credentials, processing transactions, and returning results in real time.

This paper describes a lightweight, console-based payment gateway prototype implemented in C. The system demonstrates the fundamental principles of payment processing, including user input, input validation, conditional routing, and randomized outcome simulation to mimic real-world network behavior.

While the implementation is simplified for educational purposes, it illustrates key concepts that underpin production-grade payment infrastructure. The authors aim to provide a clear analysis of the code structure, its strengths, and its limitations, along with suggestions for secure deployment.

II. SYSTEM ARCHITECTURE

The payment gateway system is organized as a modular C program comprising four functional units: the main entry point, the UPI payment module, the card payment module, and the payment processing engine.

IV. CONTROL FLOW

The control flow of the system follows a top-down sequential model. Upon execution, the user is prompted to enter a payment amount and select a payment method. The selected branch invokes the corresponding validation function. If validation succeeds, control is transferred to the processPayment() function, which returns either a success or failure message.

Figure 1 describes the logical flow of the system from initiation to transaction outcome.

```
[START] → Enter Amount → Select Method
  ↓   UPI Path          ↓   Card Path
Validate '@'      Validate len(card/cvv)
        ↓                     ↓
processPayment() → [SUCCESS / FAIL]
```

Fig. 1. System Control Flow Diagram

V. SECURITY ANALYSIS

Although the system fulfills its demonstrative purpose, several security vulnerabilities are identified:

1) Buffer Overflow Risk: The use of scanf("%s", upi) without a field-width specifier allows inputs exceeding the buffer size (50 bytes), creating potential stack smashing vulnerabilities.

2) Weak Random Number Generation: The use of rand() seeded by time(0) is not cryptographically secure and

© 2024 IJCSER — All rights reserved

The primary modules are as follows:

• main(): Handles user prompts for the payment amount and payment method selection using a switch-case control structure.

• upiPayment(): Accepts a UPI identifier from the user, validates the presence of the '@' symbol as the domain separator, and routes valid inputs to the processing engine.

• cardPayment(): Captures a 16-digit card number and a 3-digit CVV, validates their lengths using strlen(), and forwards valid credentials to the processing engine.

• processPayment(): Simulates transaction processing by introducing a computation-based delay and invoking the C standard library's rand() function seeded by the system clock to produce a binary success/failure outcome.

III. IMPLEMENTATION DETAILS

The system is implemented using ANSI C standard libraries: stdio.h for input/output, stdlib.h for random number generation, string.h for string operations, and time.h for seeding the random generator.

A. UPI Validation

UPI IDs are validated using the strchr() function to confirm the presence of the '@' character. This replicates the structural requirement of UPI identifiers such as user@bank. An example validation snippet is shown below:

```
if(strchr(upi, '@') != NULL) {
    printf("Valid UPI ID\n");
    processPayment(amount);
}
```

B. Card Credential Validation

Card numbers are validated for a length of 16 digits, and CVV codes for 3 digits, consistent with standard ISO/IEC 7813 card specifications. The validation is performed as follows:

```
if(strlen(card) == 16 &&
   strlen(cvv) == 3) {
    processPayment(amount);
}
```

C. Transaction Processing Simulation

The processPayment() function uses a busy-wait loop to simulate network latency, followed by a pseudo-random binary outcome. The random generator is seeded with time(0) to vary outcomes across executions:

```
srand(time(0));
int success = rand() % 2;
```

can be predicted if the seed value is known. A production system should use cryptographically secure pseudo-random number generators (CSPRNG).

3) Plain-text Credential Handling: Card numbers and CVV codes are stored in unencrypted stack-allocated character arrays without any masking, making them vulnerable to memory inspection.

4) No Luhn Validation: The implementation does not apply the Luhn algorithm to verify card number authenticity, which is a standard check in real payment systems.

VI. PROPOSED IMPROVEMENTS

The following enhancements are recommended for a production-ready implementation:

• Replace scanf("%s") with scanf("%49s") or fgets() to prevent buffer overflows.

• Implement the Luhn algorithm for card number verification.

• Use OpenSSL or platform-native secure RNG APIs in place of rand().

• Integrate TLS/SSL for network communication if extended to a client-server model.

• Mask sensitive input (CVV, card numbers) during entry using platform-specific terminal control.

VII. CONCLUSION

This paper presented the design, implementation, and analysis of a simple C-based payment gateway system. The system successfully demonstrates input routing, basic credential validation, and simulated transaction outcomes using core C standard library functions.

The identified security limitations highlight the gap between educational prototypes and production-grade payment systems, providing a roadmap for improvement. This work establishes a conceptual foundation for students and early-career developers seeking to understand the internals of payment processing pipelines in systems programming contexts.

REFERENCES

[1] B. W. Kernighan and D. M. Ritchie, The C Programming Language, 2nd ed. Englewood Cliffs, NJ: Prentice Hall, 1988.

[2] National Payments Corporation of India (NPCI), "Unified Payments Interface (UPI) Product Overview," 2023. [Online]. Available: https://www.npci.org.in/what-we-do/upi

[3] ISO/IEC 7813:2006, "Identification cards — Financial transaction cards," International Organization for Standardization, Geneva, Switzerland, 2006.

© 2024 IJCSER — All rights reserved

[4] D. Eastlake and T. Hansen, "US Secure Hash Algorithms," RFC 6234, Internet Engineering Task Force (IETF), 2011.

[5] Open Web Application Security Project (OWASP), "Input Validation Cheat Sheet," 2023. [Online]. Available: https://cheatsheetseries.owasp.org

© 2024 IJCSER — All rights reserved